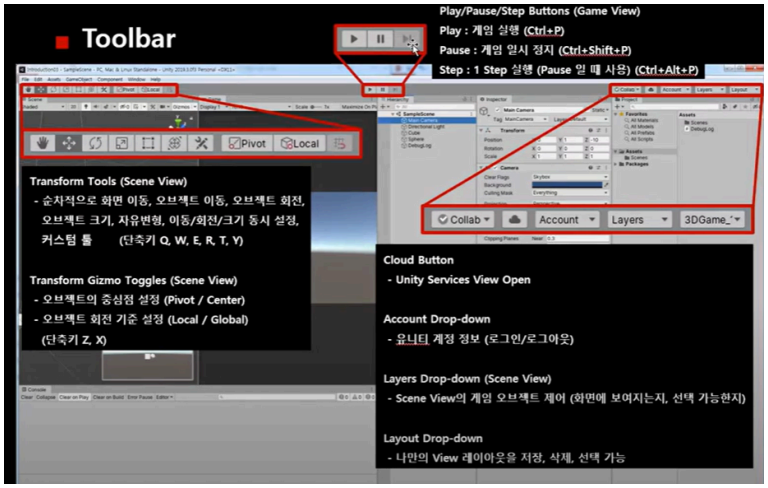
박선영 고박사 세션 1

☑ 참여 여부	☑

1. 유니티 에디터 인터페이스

- file: scene/project 생성, 삭제
- build: 프로젝트 빌드 가능
- edit: 프로젝트 설정
- esset: 패키지 파일 생성, 삭제
- Game object

- component: game object에 붙여서 사용하는 속성
- window: view를 열 수 있는 메뉴
- tool bar



- scene view: 현재 프로젝트에 존재하는 모든 게임 오브젝트를 그래픽으로 표현한 곳

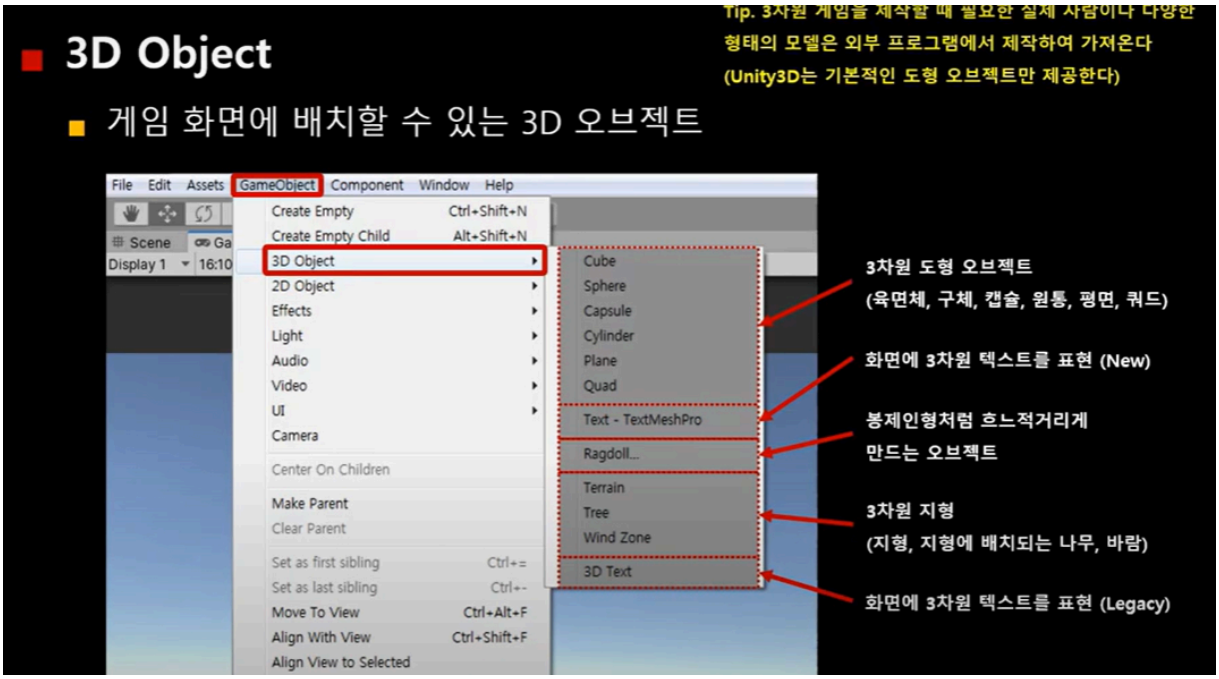
■ Scene View 화면(카메라) 조작법

동작		명령 키
화면 이동		마우스 휠 버튼 클릭 후 드래그
화면 회전		Alt + 마우스 왼쪽 버튼 클릭 후 드래그 마우스 오른쪽 버튼 클릭 후 드래그
확대 / 축소		Alt + 마우스 오른쪽 버튼 클릭 후 드래그 마우스 휠 버튼 앞/뒤 드래그
비행 모드	전, 후, 좌, 우	마우스 오른쪽 버튼 + W, A, S, D (or 방향키)
	상승/하강	마우스 오른쪽 버튼 + Q, E
	빠른 이동	Shift + "전, 후, 좌, 우" or "상승/하강"
선택된 오브젝트로 화면 이동		F

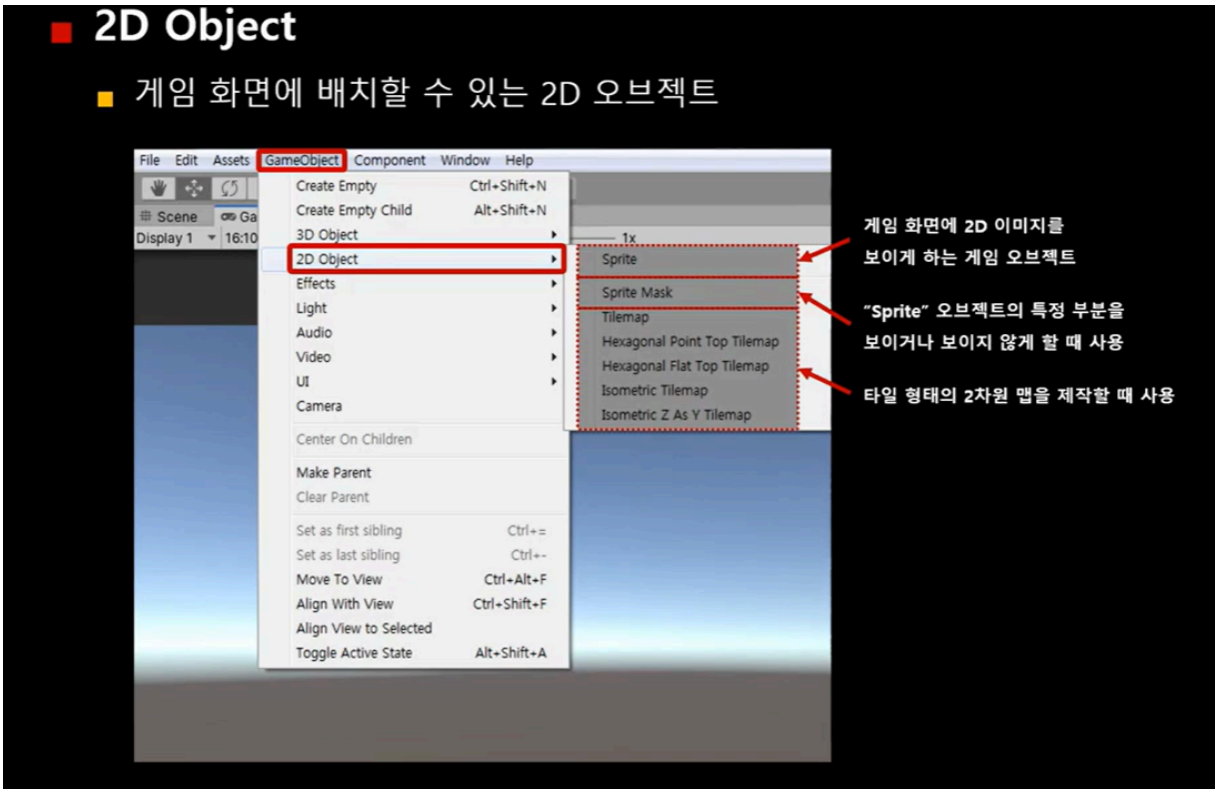
- game view: 화면에 보여지는 오브젝트들 표현한 곳. 게임 실행했을 때 화면 확인 가능
- inspector view: 오브젝트들의 상세 정보를 확인 가능
- hierarchy view: 오브젝트들을 이름으로 표현한 곳. 오브젝트들이 계층 구조를 이룰 수 있음
- project view: 프로젝트에 필요한 리소스를 볼 수 있는 곳
- console view: 코드 오류 났을 때 표시하는 곳

2. 게임 오브젝트, 컴포넌트

- sprite renderer 컴포넌트: 2차원의 이미지를 화면에 출력
- mesh renderer 컴포넌트: 3차원의 오브젝트를 화면에 출력
- audio source 컴포넌트: audioclip 변수에 지정된 오디오 출력
- 컴포넌트→C# 스크립트
- 에셋: 오디오, 모델 같은 리소스들
- 프리팹: 게임 오브젝트를 파일 형태로 저장하는 행위
- 빈 오브젝트: Transform 컴포넌트만 붙어있는 오브젝트. Transform 컴포넌트로 이동/회전/크기 제어 가

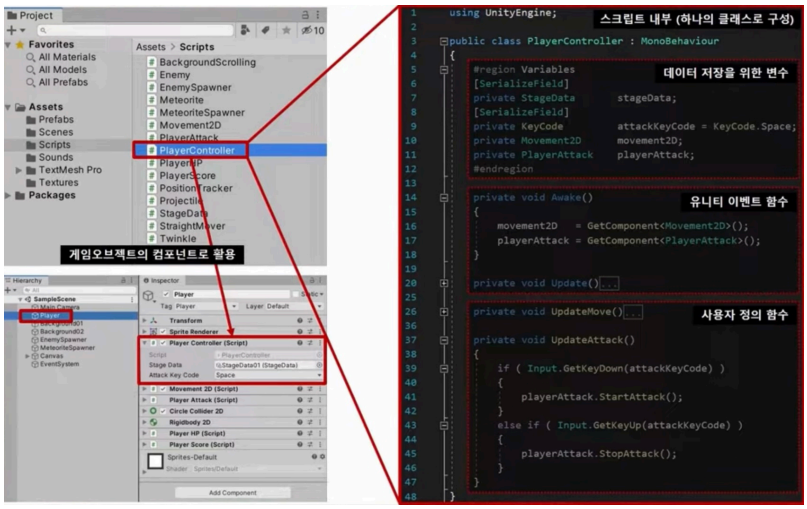


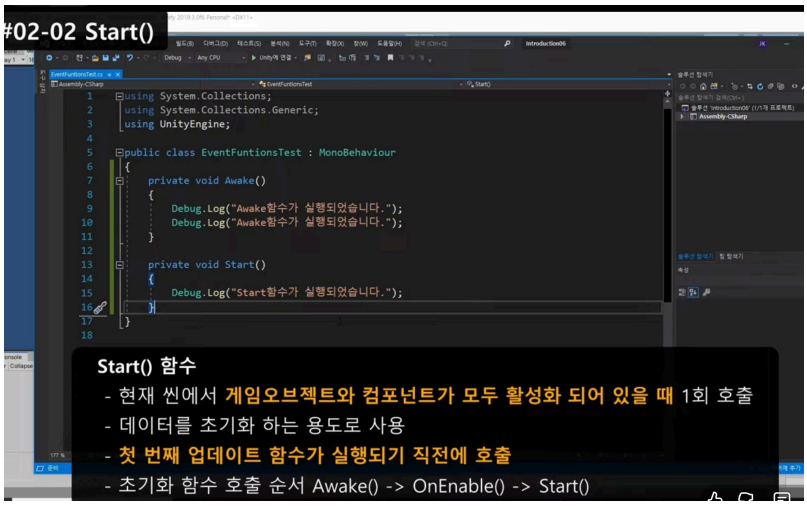
- Mesh Filter 컴포넌트: 3차원 오브젝트의 외형
- Mesh Renderer 컴포넌트: 3차원 오브젝트의 표면 색상
- Box Colider 컴포넌트: 게임 오브젝트의 충돌 범위 설정

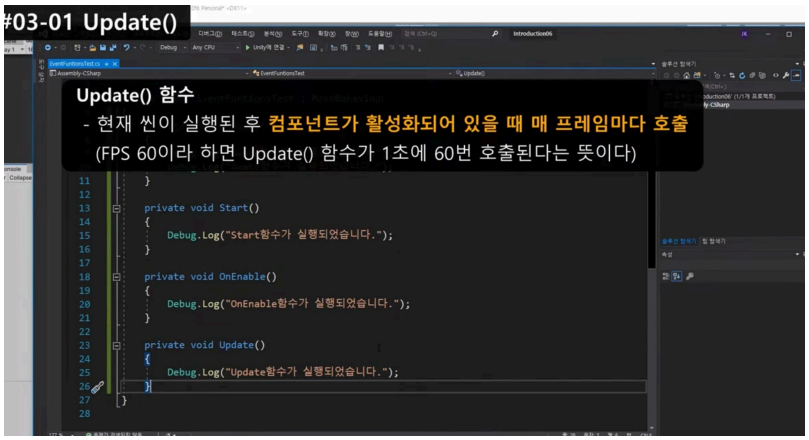


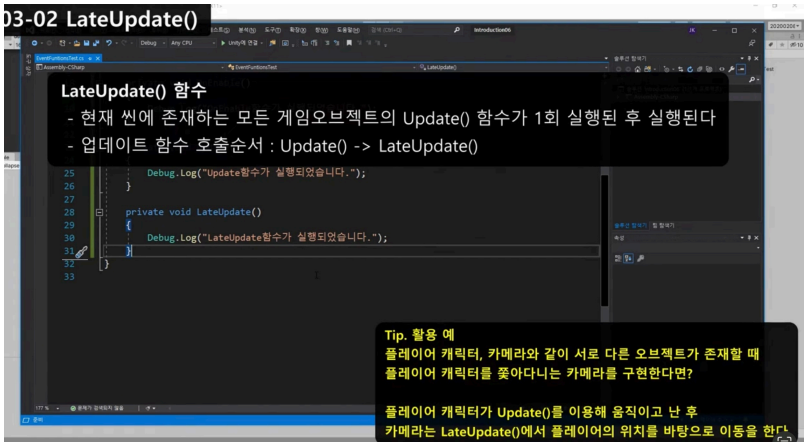
- Sprite Renderer 컴포넌트: 게임 화면에 2D 이미지를 출력
- Effect: 게임에서 사용되는 여러 효과 제공
- UI 사용자가 게임과 상호작용 할 수 있는 GUI 오브젝트→textmesh pro 사용해야 함

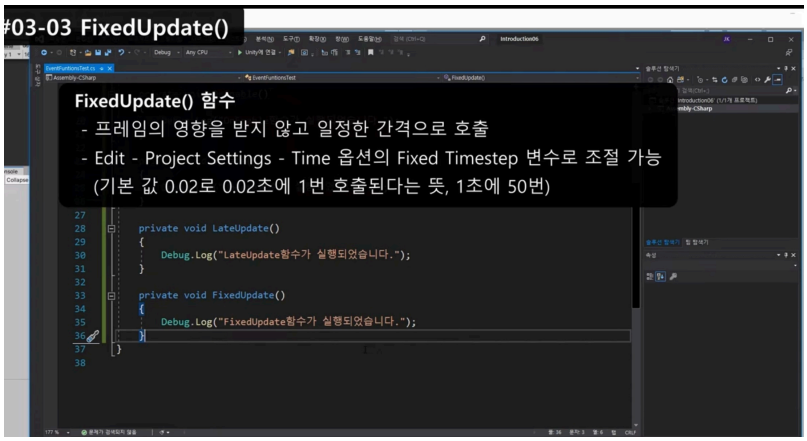
3. 스크립트

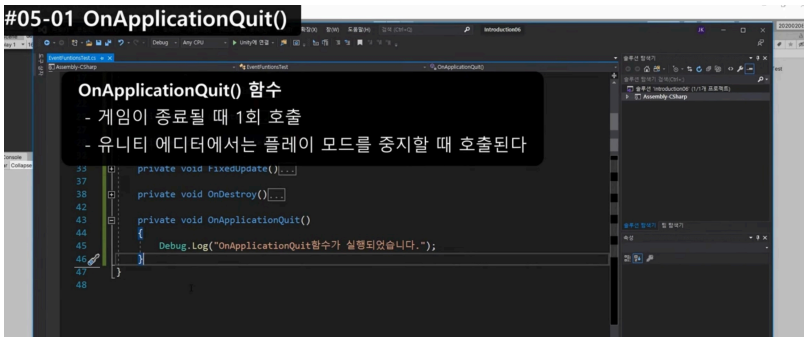



#02-02 Start()
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
public class EventFuntionsTest : MonoBehaviour
{
private void Awake()
{
Debug.Log("Awake함수가 실행되었습니다.");
Debug.Log("Awake함수가 실행되었습니다.");
}
private void Start()
{
Debug.Log("Start함수가 실행되었습니다.");
}
}
Start() 함수
- 현재 씬에서 게임오브젝트와 컴포넌트가 모두 활성화 되어 있을 때 1회 호출
- 데이터를 초기화 하는 용도로 사용
- 첫 번째 업데이트 함수가 실행되기 직전에 호출
- 초기화 함수 호출 순서 Awake() -> OnEnable() -> Start()


#03-01 Update()
Update() 함수
- 현재 씬이 실행된 후 컴포넌트가 활성화되어 있을 때 매 프레임마다 호출
(FPS 60이라 하면 Update() 함수가 1초에 60번 호출된다는 뜻이다)
}
private void Start()
{
Debug.Log("Start함수가 실행되었습니다.");
}
private void OnEnable()
{
Debug.Log("OnEnable함수가 실행되었습니다.");
}
private void Update()
{
Debug.Log("Update함수가 실행되었습니다.");
}
}


03-02 LateUpdate()
LateUpdate() 함수
- 현재 씬에 존재하는 모든 게임오브젝트의 Update() 함수가 1회 실행된 후 실행된다
- 업데이트 함수 호출순서 : Update() -> LateUpdate()
Debug.Log("Update함수가 실행되었습니다.");
}
private void LateUpdate()
{
Debug.Log("LateUpdate함수가 실행되었습니다.");
}
}
Tip. 활용 예
플레이어 캐릭터, 카메라와 같이 서로 다른 오브젝트가 존재할 때
플레이어 캐릭터를 쫓아다니는 카메라를 구현한다면?
플레이어 캐릭터가 Update()를 이용해 움직이고 난 후
카메라는 LateUpdate()에서 플레이어의 위치를 바탕으로 이동을 한다


#03-03 FixedUpdate()
FixedUpdate() 함수
- 프레임의 영향을 받지 않고 일정한 간격으로 호출
- Edit - Project Settings - Time 옵션의 Fixed Timestep 변수로 조절 가능
(기본 값 0.02로 0.02초에 1번 호출된다는 뜻, 1초에 50번)
private void LateUpdate()
{
Debug.Log("LateUpdate함수가 실행되었습니다.");
}
private void FixedUpdate()
{
Debug.Log("FixedUpdate함수가 실행되었습니다.");
}
}


#05-01 OnApplicationQuit()
OnApplicationQuit() 함수
- 게임이 종료될 때 1회 호출
- 유니티 에디터에서는 플레이 모드를 중지할 때 호출된다
private void FixedUpdate()...
private void OnDestroy()...
private void OnApplicationQuit()
{
Debug.Log("OnApplicationQuit함수가 실행되었습니다.");
}
}